

Using Databases with Render

 [_ \(https://github.com/learn-co-curriculum/python-p4-using-databases-with-render\)](https://github.com/learn-co-curriculum/python-p4-using-databases-with-render)  [_ \(https://github.com/learn-co-curriculum/python-p4-using-databases-with-render/issues/new\)](https://github.com/learn-co-curriculum/python-p4-using-databases-with-render/issues/new)

Learning Goals

- Review the process for creating a PostgreSQL instance on Render.
- Use PSQL to execute database commands.
- Create multiple databases in a PostgreSQL instance.
- Use seed data with a database.
- Back up and restore our apps' databases.
- Reset an existing database.

Introduction

In the last lesson, we learned how to deploy a Flask API on Render, including a database. We touched on some issues with Render's free tier, and on using PSQL to run some of the database commands. In this lesson, we will review some of that material and go into greater detail about using databases with Render.

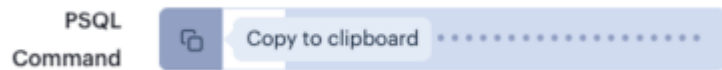
We recommend that as you go through this lesson, you pay attention to what you can do in Render and what you need to use PSQL to accomplish. In particular, PSQL will allow you to interact with any app-specific databases you add within your PostgreSQL instance. Your goal for this lesson should be to gain a basic understanding of the commands and utilities covered. You can return to this lesson and use it as a reference as needed moving forward.

Using PSQL to execute database commands

Render makes it pretty easy to accomplish certain database tasks, but there are a number of additional helpful tasks that can't be done through the browser interface. For these, we'll use the PostgreSQL interactive terminal, `psql` [↗\(https://www.postgresql.org/docs/current/app-psql.html\)](https://www.postgresql.org/docs/current/app-psql.html).

PSQL is a very helpful tool that allows us to connect to the remote database from our terminal and run certain **"meta-commands"** [↗\(https://www.postgresql.org/docs/9.2/app-psql.html#APP-PSQL-META-COMMANDS\)](https://www.postgresql.org/docs/9.2/app-psql.html#APP-PSQL-META-COMMANDS) that take the place of running a SQL query (e.g., list our databases; list the data tables in a database). It also includes some useful "utility" commands (e.g., backup a database; restore a database). Finally, we can run SQL commands directly from PSQL.

To get into PSQL, go to your PostgreSQL instance page in the Render dashboard, scroll down to the "Connections" section, and copy the "PSQL Command".



Alternatively, you can click "Connect" in the upper right corner, then click "External Connection" and copy the command from there.

Go to your terminal, paste in the command and press enter. You should now see the `psql` command prompt, `my_database=>`.

Note: As you're working with PSQL, be aware that if it's idle for a while, the connection will time out. If that happens, when you try to run a command, there will be a delay and then you'll see the following message:

```
could not receive data from server: Operation timed out
SSL SYSCALL error: Operation timed out
```

To reconnect, run the quit command (`\q`) then, from the terminal prompt, press the up arrow to access the PSQL command and hit enter.

Listing Databases

In the previous lesson, you created a PostgreSQL instance `my_database` to store all of your apps' databases. The free tier only allows you to create one database server instance, although you were able to create another database named `bird_app_db` within it by using the `CREATE DATABASE` command. Let's explore some other PSQL commands.

To list the databases on your PostgreSQL instance, run the `\l` meta-command. You should see something like this:

```
my_database=> \l
```

List of databases						
Name	Owner	Encoding	Collate	Ctype	Access privileges	
bird_app_db	my_database_user	UTF8	en_US.UTF8	en_US.UTF8		
my_database	my_database_user	UTF8	en_US.UTF8	en_US.UTF8		
postgres	postgres	UTF8	en_US.UTF8	en_US.UTF8		
template0	postgres	UTF8	en_US.UTF8	en_US.UTF8	=c/postgres	+
					postgres=CTc/postgres	
template1	postgres	UTF8	en_US.UTF8	en_US.UTF8	=c/postgres	+
					postgres=CTc/postgres	

```
(5 rows)
```

Here you can see a list of databases, including some that were created automatically by Postgres (the ones with `postgres` as the owner), and some that were created by the user. The `my_database` database is the name of the PostgreSQL instance; you can tell because the associated user (`my_database_user`) is the owner of all of the user-created databases. The remaining user-created databases are associated with particular apps. (See the "Creating Multiple Databases within a PostgreSQL Instance" section below for instructions on how to create app-specific databases under the umbrella of your PostgreSQL instance.)

Listing Data Tables

To list the data tables in the currently selected database (in our case, `my_db_instance`), run:

```
\dt
```

Because we don't have any data tables associated with our PostgreSQL instance directly, we get the following:

```
Did not find any relations.
```

To see the tables associated with one of our apps, we first need to switch to the app's database.

Switching to a Different Database

To switch to the `bird_app_db` that we created in the last lesson, we'll type the command `\c bird_app_db` :

```
my_database=> \c bird_app_db
```

You should see something similar to:

```
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_128_GCM_SHA256, bits: 128, compression: off)
You are now connected to database "bird_app_db" as user "my_database_user".
bird_app_db=>
```

The `c` stands for connect. The PSQL command prompt will now show `bird_app_db` rather than `my_database` .

Now if we enter the command `\dt` again, we'll see something like this:

```
bird_app_db=> \dt
              List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | alembic_version | table | my_database_user
public | birds           | table | my_database_user
(2 rows)
```

Note that we can see our `birds` table in the second row.

Quitting PSQL

To exit PSQL and get back to your terminal prompt, run the `\q` command.

Using Seed Data with a Database

You may recall that in the last lesson, in order to deploy our app, we created the bulk of our data with a seed file. We ran `python seed.py` to seed the database. Whenever you add the seed data, just push up the changes to GitHub and re-deploy your app. Recall that seed scripts should begin with a deletion of existing data to avoid duplicating the data.

If you forget to do that, or if you need to start fresh for some other reason, you can reset the database. We'll learn how to do that a bit later in this lesson.

Backing Up and Recreating Your Databases on Render

We mentioned in the last lesson that, with Render's free tier, your PostgreSQL instance and any additional databases you've created on it will expire after 90 days. This means that, before the end of the 90 days, you will need to back up your databases, delete the PostgreSQL instance from Render, create a new PostgreSQL instance, and populate it from the database backups. Render should send an email warning you that your database will be expiring soon.

Before we launch into the instructions for backing up and recreating your Render PostgreSQL instance, let's take a closer look at the PSQL connection string. Go ahead and copy the "PSQL Command" from the Render dashboard then paste it into a new file in your text editor.

Warning: Be careful not to store any of the PSQL commands inside a project repo. Those commands contain secure information so you don't want them to be deployed to GitHub accidentally!

The connection string will look something like this:

```
PGPASSWORD=##### psql -h #####-postgres.render.com -U my_database_user my_database
```

The first element is the password for your database, which will be a 32-character string. Next is the command you're running, in this case, `psql`. The next component is the host (indicated by the `-h` flag), which will end with `-postgres.render.com`. Next is the name of the database user (indicated by the `-U` flag), followed, finally, by the name of the database itself.

Note that the username and database name in the PSQL command above match the entry for the PostgreSQL instance in the list of databases we printed earlier in this lesson.

We discovered earlier that the main database in the PostgreSQL instance, `my_database`, does not contain any data. Therefore, we just need to back up the database we've created, `bird_app_db`.

Note: Technically, we don't need to back up our bird app either, because all the records come from our seed data — there is no way for users to add, change or delete records. This means all we need to do to restore the database is re-run the seed command. For purposes of illustration, however, we'll go ahead and go through the process of backing it up.

Backing Up Our Databases

To create a backup, we're going to modify the PSQL connection string to run PSQL's `pg_dump` [↗\(https://www.postgresql.org/docs/15/app-pgdump.html\)](https://www.postgresql.org/docs/15/app-pgdump.html) utility command. We'll do that once for each database we need to back up, starting with `bird_app_db`. We recommend making the edits to the string in your text editor then copy/pasting it into the terminal when you're done. (But remember not to push it up to Github!)

The first part of the string, the password, will remain the same. The `psql` command should be updated to `pg_dump` instead. The host and username should also stay the same. After that, we'll add the following options:

```
--format=custom --no-acl --no-owner
```

The final component of the original connection string is the name of the PostgreSQL instance, `my_database`. We'll replace that name with `bird_app_db` instead. After that, we'll add a `>` to indicate that we want the results of the command to be written to a file, followed by the name we want to use for the backup file, with the `.sql` extension:

```
bird_app_db > bird_app_db.sql
```

The updated string will look something like this:

```
PGPASSWORD=##### pg_dump -h #####-postgres.render.com -U my_database_user --format=custom --no-acl
```

Now that we've created the command, we'll copy/paste it into the terminal (not PSQL) and run it. Make sure you're not inside a directory that's a git repo first to ensure the backup file doesn't get pushed to GitHub. The command will not print any output, but if we run `ls`, we'll see the newly-created `.sql` file in the current directory.

Now that we've backed up our database, the next step is to delete the current PostgreSQL instance and create a new one.

Replacing the Expiring PostgreSQL Instance

To delete the PostgreSQL instance, go to the database page on the Render dashboard. Scroll to the bottom of the page, then click "Delete Database" and follow the instructions.

Next, we'll create a new PostgreSQL instance by clicking the "New +" button and selecting PostgreSQL. Provide a name for the new instance (e.g., `my_database`), then scroll down to the bottom of the page, and click "Create Database."

You need to copy the PSQL connection string from this database instance, since it is different from the previous version.

Restoring the Databases to the New Instance

Once the new instance has been created, the next step is to create the app-specific databases within that instance.

First we need to execute the PSQL connection string for the new PostgreSQL instance to launch the interactive terminal. Next, we'll run the CREATE DATABASE commands for each of the databases (we're using the same database names but you can use different names if you prefer):

```
my_database=> CREATE DATABASE bird_app_db;
```

Once that's done, we can exit PSQL with the `\q` command.

Now we're ready to work on building the `pg_restore`  (<https://www.postgresql.org/docs/15/app-pgrestore.html>) command. Once again, we recommend pasting the connection string into your text editor and editing it there.

The command will consist of the following:

- The database password.

- The `pg_restore` command.
- The host.
- The user.
- The options: `--verbose --clean --no-acl --no-owner` .
- The `-d` flag (for `dbname`) followed by the name of the new database you're restoring the data to.
- The name of the `.sql` file you're restoring from.

The final string for the bird app will look something like this:

```
PGPASSWORD=##### pg_restore -h #####-postgres.render.com -U my_database_user --verbose --clean
```

When we run the command in the terminal, we'll see a flurry of activity as it creates the database tables. You may see some error messages about dropping tables, which you can ignore.

Connecting the New Databases to Your Web Services

The final step in the process is to update each app's Web Service so that it points to the newly-restored database.

From the Render dashboard, we'll select the bird app, then click "Environment" in the nav on the left. Next, we delete the value associated with the `DATABASE_URL` key and replace it with the Internal URL for the new instance. Remember that we want to connect to the bird app database, not the instance itself, so you need to remove the name of the PostgreSQL instance from the end of the URL and replace it with the name of the bird app database. You also need to update the protocol to `postgresql` . It should look something like this:

```
postgresql://my_database_user:#####/bird_app_db
```

After we've saved the change, the web service will redeploy. When we click the app's URL for the `/birds` route, we should see the JSON for the list of birds.

Resetting an Existing Database

If you need to reset a database for some reason — say you have duplicate data — all you need to do is drop the database and recreate it.

There are two cases to consider. The first is if the database you need to reset is the PostgreSQL instance itself (i.e., if you only have a single app deployed). In this case, you would simply delete and recreate the instance in Render, then connect the new instance to the web service for the app and redeploy it.

If, on the other hand, you need to reset a database *within* your PostgreSQL instance and not the instance itself, you can do that using PSQL:

1. Execute the PSQL command for your PostgreSQL instance in the terminal.
2. Run the SQL command to drop the database: `DROP DATABASE database_name;` . You should see 'DROP DATABASE' echoed in the terminal.
If you don't, make sure you included the semicolon and that your PSQL connection hasn't timed out.
3. Run the SQL command to create the new database: `CREATE DATABASE database_name;`
4. In Render, connect the web service for the app to the new database and redeploy. If you used the same name for the database, you'll just need to redeploy.

Optionally, with either approach, you can also re-seed your database if you choose.

That's it!

Conclusion

In this lesson, you learned some important database tasks that can be performed using the Render dashboard, and how to use PSQL to supplement those actions. In particular, familiarity with PSQL will be very helpful for you if you want to deploy multiple apps with databases to Render. You've also learned how to handle the situation when your Render PostgreSQL instance is approaching its expiration date.

In the next lesson, we'll work on deploying a more complex application with a Flask API backend and a React frontend, and talk through some of the challenges of running these two applications together.

Check For Understanding

Before you move on, make sure you can answer the following questions:

- ▶ 1. *What database actions can be completed in the Render web app?*
- ▶ 2. *What database actions require the PSQL interactive terminal?*
- ▶ 3. *What issue do you need to be aware of when using seed data for your database? How can you address the situation?*

Resources

- [Render Databases Guide](https://render.com/docs/databases)  [_\(https://render.com/docs/databases\)](https://render.com/docs/databases)
- [Multiple Databases In A Single PostgreSQL Instance](https://render.com/docs/databases#multiple-databases-in-a-single-postgresql-instance)  [_\(https://render.com/docs/databases#multiple-databases-in-a-single-postgresql-instance\)](https://render.com/docs/databases#multiple-databases-in-a-single-postgresql-instance)